

◆ INITIAL VALUE

```
String result = "ERROR";
```

Before reading Flask's response, we create a variable.

What is String?

```
String
```

is **a class** used to store text.

Example:

```
String name = "John";
```

stores:

```
John
```

Similarly:

```
String result = "ERROR";
```

stores:

```
ERROR
```

What is result?

```
result
```

is a variable.

It will eventually store:

```
SPAM
```

or

```
HAM
```

Why Start With "ERROR"?

Many beginners ask:

"Why not leave it empty?"

Good question.

Suppose Flask fails.

Suppose response body is missing.

Suppose prediction field is missing.

Then:

```
SPAM
```

or

```
HAM
```

will never be assigned.

By setting:

```
ERROR
```

first,

we always have a default value.

Think of it as:

```
Prediction Unknown
      ↓
Use ERROR
```

Example

Suppose Flask returns:

```
{
}
```

No prediction field exists.

Then result remains:

```
ERROR
```

instead of causing problems.

Safety Net

```
String result = "ERROR"; check sd pdf
```

acts as a backup value.

◆ CHECKING THE RESPONSE

```
if (response.getBody() != null &&
    response.getBody().get("prediction") != null)
```

This is one of the most important **safety checks**.

Let's break it apart slowly.

What Are We Trying To Check?

We want to safely access:

```
prediction
```

from Flask's response.

Expected Flask response:

```
{
  "prediction": "SPAM"
}
```

Before reading prediction, we must verify:

1. Response body exists.

2. Prediction exists.

Otherwise the application may crash.

◆ PART 1 : response.getBody()

```
response.getBody()
```

returns the body of the **HTTP response**.

Remember:

```
response
├── Status
├── Headers
└── Body
```

Example:

```
HTTP 200 OK

{
  "prediction": "SPAM"
}
```

Then:

```
response.getBody()
```

returns:

```
{
  prediction=SPAM
}
```

as a Map.

Visual

```
response
  ↓
Body
  ↓
{
  prediction=SPAM
}
```

◆ WHAT IS null?

This is extremely important.

```
null
```

means:

"Nothing exists."

Example: // eg bhao

```
String name = null;
```

Memory:

```
name
  ↓
nothing
```

No String object exists.

Another Example

```
EmailResponseDTO dto = null;
```

means:

```
dto
  ↓
nothing
```

The variable exists.

But no object is attached to it.

Why Is null Dangerous?

Suppose:

```
String name = null;
```

and you do:

```
name.length();
```

Java crashes.

Why?

Because there is no String object.

This causes:

```
NullPointerException
```

One of the most common Java errors.

◆ FIRST CHECK

```
response.getBody() != null
```

means:

"Does the response body actually exist?"

Case 1 : Body Exists

Flask returns:

```
{  
  "prediction": "SPAM"  
}
```

Then:

```
response.getBody()
```

contains a Map.

Condition:

```
response.getBody() != null
```

becomes:

```
true
```



Case 2 : Body Missing

Suppose Flask returns:

```
No Body
```

Then:

```
response.getBody()
```

returns:

```
null
```

Condition becomes:

```
false
```



This prevents a crash.

◆ PART 2 : get("prediction")

```
response.getBody().get("prediction")
```

Let's assume body exists.

Example:

```
{
  prediction=SPAM
}
```

Now we ask:

```
get("prediction")
```

This means:

"Give me the value stored under the key prediction."

Map:

```
{
  prediction=SPAM
}
```

Result:

```
SPAM
```

Another Example

Map:

```
{  
  name=John  
}
```

Calling:

```
get("name")
```

returns:

```
John
```

Exactly the same concept.

◆ SECOND CHECK

```
response.getBody().get("prediction") != null
```

means:

"Does prediction actually exist?"

Case 1

Response:

```
{  
  "prediction":"SPAM"  
}
```

Then:

```
get("prediction")
```

returns:

```
SPAM
```

Condition:

```
SPAM != null
```

becomes:

```
true
```

Case 2

Response:

```
{  
}
```

No prediction key.

Then:

```
get("prediction")
```

returns:

```
null
```

Condition:

```
null != null
```

becomes:

```
false
```

Again prevents crashes.

◆ WHY USE && ?

```
&&
```

means:

AND

Both conditions must be true.

Truth Table:

```
true  && true  = true  
true  && false = false  
false && true  = false  
false && false = false
```

Our Condition

```
response.getBody() != null
&&
response.getBody().get("prediction") != null
```

Meaning:

```
Body Exists?
AND
Prediction Exists?
```

Only then continue.

Why Is This Safe?

Suppose body is null.

```
response.getBody() == null
```

First condition becomes:

```
false
```

Java immediately stops.

It does NOT evaluate:

```
response.getBody().get("prediction")
```

This is called:

Short-Circuit Evaluation

Why Is That Important?

Without short-circuiting:

Java would try:

```
null.get("prediction")
```

which causes:

```
NullPointerException
```

But because of:

```
&&
```

Java safely stops early.

◆ IF THE CONDITION IS TRUE

Suppose Flask returns:

```
{
  "prediction": "SPAM"
}
```

Then:

```
response.getBody() != null
```

↓

```
true
```

and:

```
response.getBody().get("prediction") != null
```

↓

```
true
```

Final result:

```
true && true
```

↓

```
true
```

So Java enters the if block:

```
if (...) {
```

and executes the next lines.

Flow

```
Response Received
  ↓
Body Exists?
  ↓
Yes
  ↓
Prediction Exists?
  ↓
Yes
  ↓
Enter if Block
```

◆ SUMMARY

```
String result = "ERROR";
```

creates a default value.

```
response.getBody()
```

gets the response body.

```
null
```

means no object exists.

```
response.getBody() != null
```

checks whether the body exists.

```
response.getBody().get("prediction")
```

gets the prediction value.

```
response.getBody().get("prediction") != null
```

checks whether prediction exists.

```
&&
```

means both conditions must be true.

This entire condition is simply asking:

"Did Flask successfully return a prediction value?"

If the answer is yes, the code enters the `if` block and executes:

```
result =  
    response.getBody()  
        .get("prediction")  
        .toString();
```

which is the next section.